

Aula de JavaScript: Estruturas de Repetição

Este README apresenta os conceitos de repetição em JavaScript com base nos exercícios da pasta `repeticao`.

Objetivos

- Entender quando usar `while`, `do...while` e `for`
- Praticar estruturas de repetição para resolver problemas reais
- Aprender a usar `break` e `continue`
- Criar contadores, somas acumuladas e repetição aninhada

Instalação e execução

Esta pasta usa `prompt-sync` para ler dados do teclado. Para instalar dependências, execute:

```
cd repeticao
npm install prompt-sync
```

Para rodar um exercício:

```
node nome_do_arquivo.js
```

Estruturas de repetição estudadas

`while`

Usado quando a quantidade de repetições não é conhecida de antemão e a condição é verificada antes de cada iteração.

Exemplos:

- `lista1.js`: encontra o maior número em uma sequência de valores
- `decrescente.js`: imprime números de 20 a 1
- `tabuada.js`: imprime a tabuada de um número
- `soma1a50.js`: soma os números de 1 a 50

`do...while`

Semelhante ao `while`, mas garante que o bloco seja executado pelo menos uma vez, pois a condição é verificada depois.

Exemplos:

- `exemplos2.js`: menu repetido até o usuário escolher sair
- `senha.js`: pede senha enquanto não for correta
- `someNumeros.js`: lê números até o usuário digitar `-1`
- `renda.js`: lê dados de alunos até a renda pessoal ser `0`
- `trianguloAsterisco.js`: imprime um triângulo de asteriscos

for

Recomendado quando sabemos quantas vezes o loop deve repetir. É uma forma compacta de iniciar contador, testar condição e atualizar o contador.

Exemplos:

- `adivinhacao.js`: jogo de adivinhação com até 10 tentativas
- `alunos_media.js`: calcula a média de alunos com notas em um loop aninhado
- `lista2.js`: encontra o menor número em uma sequência de valores
- `mega_senha.js`: gera 6 números aleatórios
- `tabuada3.js`: imprime várias tabuadas com laços aninhados

Uso de `break` e `continue`

- `break`: sai imediatamente do laço atual
- `continue`: pula para a próxima iteração do laço

Exemplo:

- `exemplos_break_continue.js`
 - `break` é usado para sair de um `while (true)` quando a senha certa é digitada
 - `continue` é usado para pular números pares em um `for`

Exercícios da pasta

- `adivinhacao.js`: jogo de adivinhação com `for`
- `alunos_media.js`: média de notas com `for` aninhado
- `decrecente.js`: contagem regressiva com `while`
- `exemplos1.js`: contagem de 1 a 10 com `while`
- `exemplos2.js`: menu com `do...while`
- `exemplos_break_continue.js`: ilustra `break` e `continue`
- `lista1.js`: maior valor usando `while`
- `lista2.js`: menor valor usando `for`
- `mega_senha.js`: gera números com `for`
- `renda.js`: processamento de dados com `do...while`
- `senha.js`: repetição até senha correta com `do...while`
- `soma1a50.js`: soma dos números de 1 a 50 com `while`
- `someNumeros.js`: soma até ler `-1` com `do...while`
- `tabuada.js`: tabuada de um número com `while`
- `tabuada2.js`: tabuadas de 1 a 10 com `while` aninhado
- `tabuada3.js`: tabuadas com `for` aninhado

- `trianguloAsterisco.js`: triângulo de asteriscos com `do...while`

Dicas de estudo

- Compare `while` e `do...while` em situações de teste antes e depois da execução.
- Use `for` sempre que souber o número exato de repetições.
- Faça pequenas modificações nos exercícios para treinar:
 - alterar o limite de repetição
 - mudar condições de parada
 - usar `break` com diferentes valores
 - trocar `while` por `for` quando possível

Bom estudo! Pratique cada exercício e experimente variações nos laços para fixar o conceito de repetição em JavaScript.